Web Service – Task Manager

Bernacchia Alessia
Sergio Fernandez Diz

What is our web service?

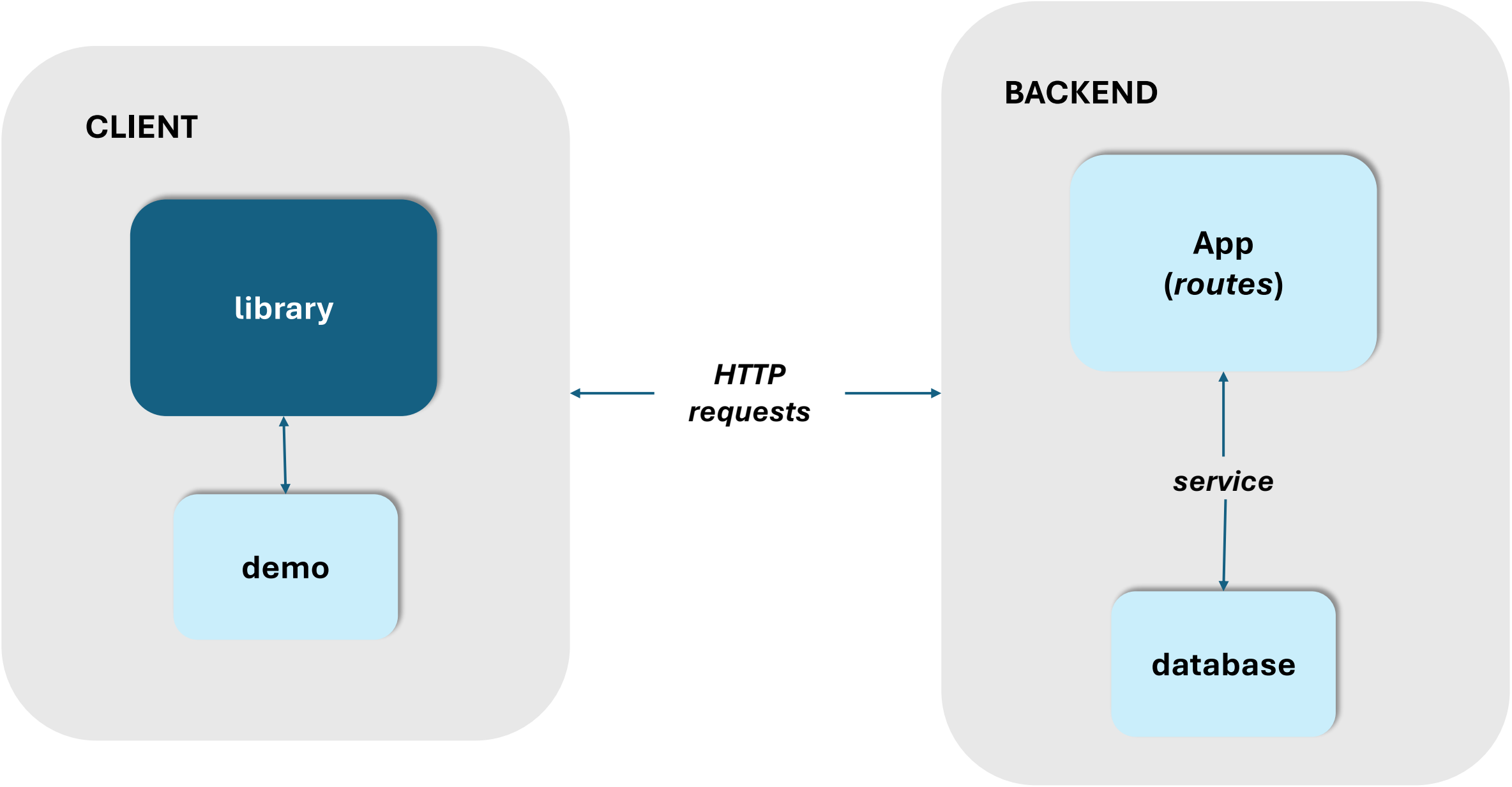
How does it interact with users and data?

What service will it provide?

Why could it be useful?

Who would benefit from it?

What problems does it solve?


CLIENT
library
demo
HTTP requests
BACKEND
App (routes)
service
database

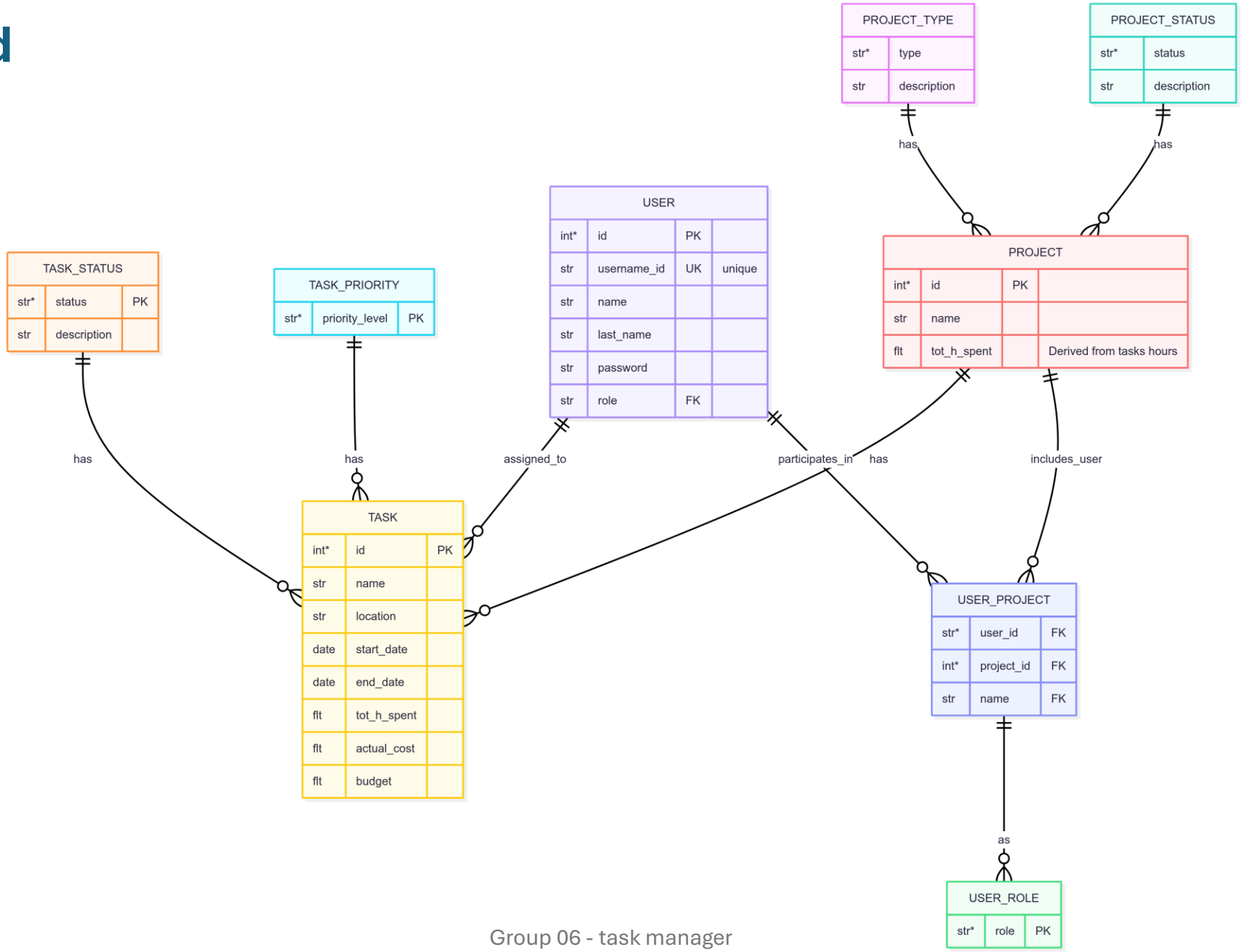
Problem	Approach	*Service*
data persistence	data storage	*SQLite*
ask queries to db	ask queries to db	*Flask App*
retrieve responses	API	*Python classes*
permit large usage	export package	*wheel*

Backend

Approach	*Service*
data storage →	*SQLite*

- lightweight
- serverless database
- widely used for small-scale

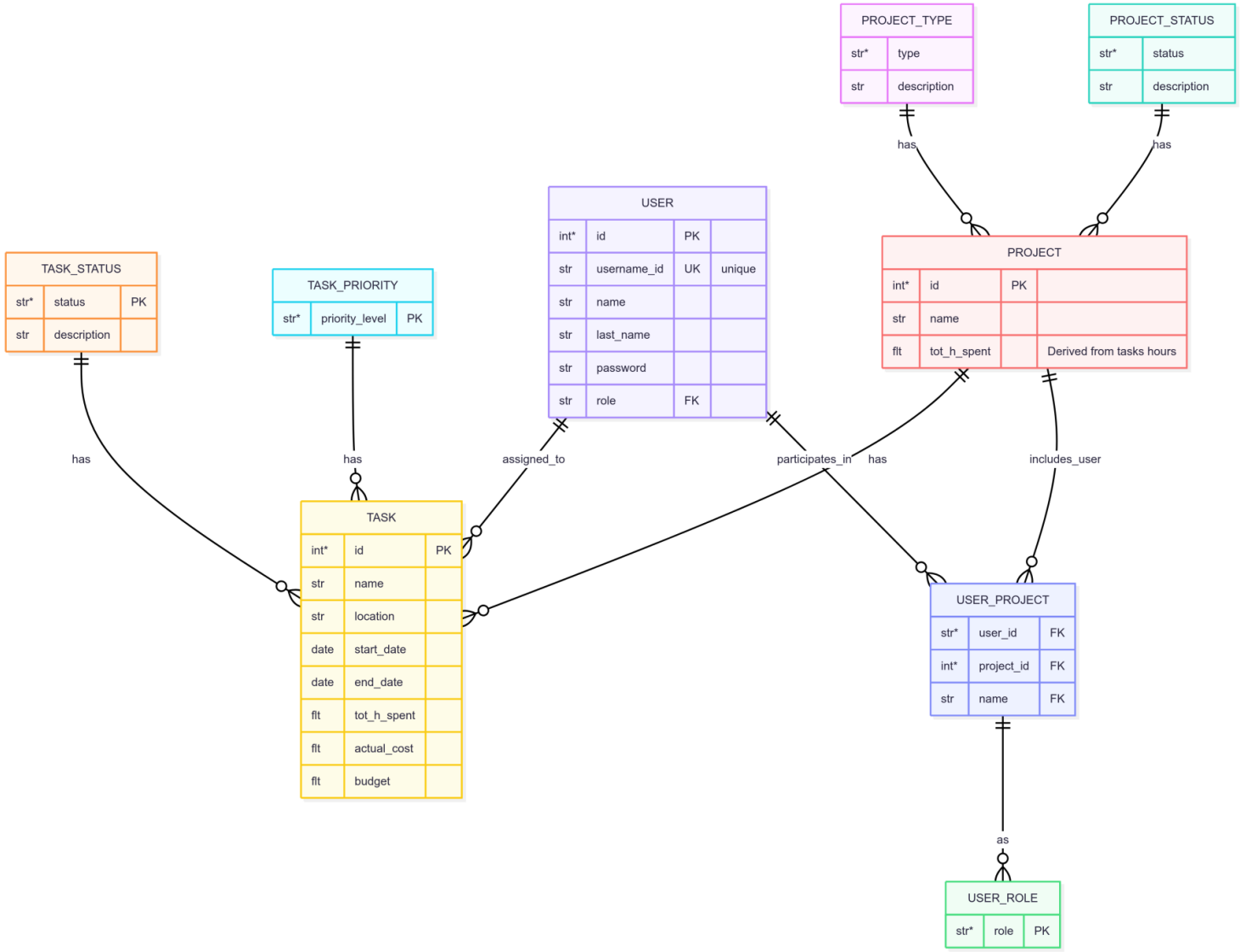
Backend


PROJECT_TYPE
str* type
str description
PROJECT_STATUS
str* status
str description
has
has
USER
int* id PK
str username_id UK unique
str name
str last_name
str password
str role FK
PROJECT
int* id PK
str name
flt tot_h_spent Derived from tasks hours
TASK_STATUS
str* status PK
str description
TASK_PRIORITY
str* priority_level PK
has
has
assigned_to
participates_in
has
includes_user
TASK
int* id PK
str name
str location
date start_date
date end_date
flt tot_h_spent
flt actual_cost
flt budget
USER_PROJECT
str* user_id FK
int* project_id FK
str name FK
as
USER_ROLE
str* role PK

Backend

We defined three strong entities
User
Project
Task

with two many-to-many relationships
User ⇄ Project
User ⇄ Task



Backend

Approach	*Service*
data storage →	*SQLite*
ask queries to db →	*Flask App*

- REST ful backend
- easy integrable in Python
- flexible
- extensible

Client

Approach	*Service*
data storage →	*SQLite*
ask queries to db →	*Flask App*
API →	*Python classes*
export package →	*wheel*

python™

- modular design
- object oriented
- reusable

for **wheel**

- **versioned**
- **easy to distribute**

Client

Project APIs	Task APIs	User APIs	Relationship APIs
get_projects()	get_tasks()	get_users()	assign_user_to_project()
get_project_by_id()	get_task_by_id()	get_user_by_id()	assign_user_to_task()
create_project()	create_task()	create_user()	remove_user_from_task()
update_project()	update_task()	update_user()	remove_user_from_project()
delete_project()	delete_task()	delete_user()	assign_project_to_user()
get_users_by_project()	get_users_by_task()	get_projects_by_user()	assign_task_to_user()
get_tasks_by_project()	get_project_by_task()	get_tasks_by_user()	remove_project_from_user()
list_projects_all_status()	list_tasks_all_status()		remove_task_from_user()
list_projects_all_types()	list_tasks_all_priorities()		

Aggregations/Statistics APIs	Interface APIs	Visualizations APIs
get_progress_by_tasks()	project_interface()	view_hours_task_project()
get_progress_by_project()	task_interface()	view_tasks_status_project()
get_total_hours_by_task()	user_interface()	view_tasks_priority_project()
get_total_hours_by_project()		show_project_summary_gui()
get_total_hours_by_user()		show_gantt_gui()
get_hours_on_project_by_user()		

python™

- modular design
- object oriented
- reusable

for **wheel**

- **versioned**
- **easy to distribute**

Approach	*Service*
data storage →	*SQLite*
ask queries to db →	*Flask App*
API →	*Python classes*
export package →	*wheel*

Backend

Client